

Readiness Standard

What every developer is expected to be able to do on Day 1, stated as things you can be seen doing – and how you show us, before the 21st, that you can.

github.com/esron-compliance-command/cc-react-lwc-starter

ISSUED

15 September 2026

EVIDENCE DUE

19 September 2026, end of day

EFFECTIVE

21 September 2026, Day 1

APPLIES TO

All developers joining the project

HOW TO READ THIS

This is a bar, not a syllabus. It does not tell you how to learn anything, and it does not care where you learn it – courses, documentation, a colleague, an afternoon of breaking things. That part is yours.

What it does do is state each expectation as something **observable**. "Be confident in React" cannot be checked by anyone, including you; "build a component with state in a custom hook that fetches on mount and cancels correctly" can. Every line below is written that way on purpose, so that nobody – you, your lead, or the person reviewing your first pull request – has to guess.

WHY WE ARE ASKING FOR EVIDENCE RATHER THAN TRUSTING THE ANSWER

Not because anyone is presumed to be overstating. Because "I know React" and "I have built this" are different claims, and only the second one predicts what happens in week one. A gap you name on 19 September is a scheduling question. The same gap discovered on 2 October is a delivery question.

THE STANDARD

Git and GitHub

DAILY, FROM HOUR ONE · NO RAMP-UP PERIOD

- Branch from the correct base, commit in coherent units, open a pull request, and respond to review comments by pushing follow-up commits.
- Resolve a merge conflict across several files, by reading both sides – not by taking one wholesale.
- Rebase your branch onto a moved base branch and explain what that did to your commits.
- Recover from a mistake with `git stash` and `git reflog` without asking anyone to rescue the branch.
- Establish, before making a claim about a commit, whether your local refs are current.

HOW THIS BECOMES EVIDENT

Immediately, and to everyone. Branch hygiene and conflict handling are visible in every pull request you open.

Lightning Web Components

THE SURFACE MOST OF OUR USERS ACTUALLY TOUCH

- Explain what runs in `connectedCallback`, `renderedCallback` and `disconnectedCallback`, and why `renderedCallback` needs a guard.
- Pass data down with `@api`, send it back up with a custom event, and choose correctly between that and Lightning Message Service for two components that are not related.
- Call Apex both imperatively and through a wire, and say which one a given situation wants.
- Load a third-party JavaScript library from a static resource with `platformResourceLoader`, into a node marked `lwc:dom="manual"`.
- Design a component so a second module can reuse it without being modified.

HOW THIS BECOMES EVIDENT

Task 2 below, and the shape of your first component in review.

React

INDEPENDENTLY, NOT WITH A TUTORIAL OPEN

- Build a component from nothing: props, state, events, conditional rendering, lists with correct keys, controlled inputs.
- Use `useState`, `useEffect` with a cleanup function, `useCallback` and `useMemo`, and justify each one you reach for.
- Move state out of a component into a custom hook, and say why that made it testable.
- Update state immutably, and explain why a mutated array does not re-render.
- Explain what StrictMode's double render is telling you, rather than suppressing it.

HOW THIS BECOMES EVIDENT

Tasks 3 and 4. Arriving able to name the hooks but not to build with them is the specific failure this section exists to prevent.

React running inside LWC

OUR ARCHITECTURE · THE PART NO TUTORIAL COVERS

- Explain why React code cannot `import` from `@salesforce/apex`, and what the LWC host does instead.
- Describe the mount contract: configure the bridge, mount into a manual DOM node, unmount on disconnect – and what breaks if you get that order wrong.
- Say why the bundle must carry React inside it rather than loading from a CDN.
- Say why the bridge is what makes the React side testable without an org.

HOW THIS BECOMES EVIDENT

Task 1, which is deliberately the first one. This is the single highest-value thing on this page.

TypeScript

READ IT, AND WRITE IT – OUR REACT CODE IS .TSX THROUGHOUT

- Read a generic signature aloud and say what it guarantees.
- Write interfaces, type aliases, unions and a generic of your own; use `import type` correctly.
- Work under `strict` without reaching for `any` to make an error go away.

HOW THIS BECOMES EVIDENT

Task 4, and every review: `any` is a review comment here, not a shortcut.

Apex, and the patterns this codebase uses

SEPARATION OF CONCERNS IS NOT NEGOTIABLE IN REVIEW

- Keep queries in a selector, business rules in a service, and neither in a controller or a trigger body.
- Explain the selector, service, domain and unit-of-work patterns and what each is for.
- Recognise a common trigger-framework shape and work inside one.
- Write a test that asserts *behaviour*. A test that runs a method and checks it did not throw is not coverage of anything.
- Write bulk-safe code: no SOQL or DML inside a loop, ever.

READ THIS BEFORE YOU STUDY FFLIB

This codebase contains **no fflib classes at all**. The same separation is implemented in-house: an `Application` singleton handing out services and selectors, an abstract `ASelector` base, around 64 selector classes, service classes, and trigger handlers behind 144 triggers. Learn the *patterns* – they are exactly what review will hold you to. Do not spend the week memorising a library API that this project never calls.

Security and sharing

OUR USERS ARE CUSTOMERS ON A PUBLIC-FACING SITE

- Choose between `with sharing`, `without sharing` and `inherited sharing`, and justify the choice.
- State what `USER_MODE` enforces and what it does *not* – it governs object and field access, not record sharing, and the two fail in different ways.
- Write a dynamic query that cannot be made to query a field the caller invented.
- Say why a new field is invisible to everyone, including an administrator, until it is in a permission set.

HOW THIS BECOMES EVIDENT

Task 5, which asks you to make the system refuse you.

Experience Cloud and LWR

WHERE EVERYTHING WE BUILD IS ACTUALLY SEEN

- Explain what an LWR site is and how a component gets onto a page in Experience Builder.
- Say why a successful deploy can still leave a customer seeing nothing at all.
- Describe the publish step, and what happens if you skip it.

HOW THIS BECOMES EVIDENT

Task 6.

Engineering judgement

APPLIED TO YOUR OWN CODE, IN REVIEW – NOT RECITED

- Recognise the second time you write something and make it the first reusable thing instead.
- Look at a method and say honestly how many branches deep it is, and what it does that its name does not say.
- Build the thing that was asked for, not the framework it might one day need.
- Be comfortable in VS Code for Salesforce work, Git, and navigating a codebase you did not write.

HOW THIS BECOMES EVIDENT

In every review, continuously. Nobody will ask you to define SOLID; you will be asked why a class has four reasons to change.

HOW YOU DEMONSTRATE IT

There is a small, complete, working reference implementation of our architecture at github.com/esron-compliance-command/cc-react-lwc-starter: a React and TypeScript grid, bundled and hosted inside an LWC, backed by one Apex controller that is generic over the object. Clone it, deploy it to your own free Developer org, and complete the six tasks below.

The tasks are not a course. They are the smallest set of things that cannot be done by someone who is not ready.

01 Deploy it, and explain the bridge

Get the grid running in your own org and put the component on a page. Then write three or four sentences explaining why the React code cannot import Apex directly and what the LWC host does about it.

PASS

A screenshot of the grid showing your own records, plus your explanation in your own words.

02 Add a third object without writing code

Make the same grid list Opportunities. No new Apex class, no new LWC, no React change.

PASS

A screenshot, and the diff – which should contain configuration only.

03 Add a feature to the React side

Add a control that lets the user hide and show columns. Notice whether it needed an Apex change, and be ready to say why not.

PASS

Working UI, and your answer to the "why not" question.

04 Extend the contract end to end

Add a server-side filter – Industry, say. That means touching the TypeScript request type, the Apex request class and the query, and adding a test on each side. One commit.

PASS

The commit, with `tsc` clean, React tests green, and the Apex test green.

05 Make the system refuse you

Remove a field from your permission set and reload the grid. Explain what happened, at which layer, and why that is the correct behaviour rather than a bug.

PASS

The error you saw, and your explanation of which gate stopped it.

06 Put it on a site and test it like a user

Create an LWR site, place the component on a page, publish, and write one Playwright test that drives the real page and passes.

PASS

The passing test output, and the site URL.

ON THE AMOUNT OF HELP AVAILABLE

The starter's README explains the architecture and documents the traps, including three that only appear when you deploy to a real org. Using it is expected – reading a reference implementation is what engineers do. What is not expected is arriving on the 21st having read it and built nothing.

WHAT YOU RETURN, AND WHEN

Send this back by **end of day Friday 19 September**. It takes a few minutes if you have done the work.

AREA	READY	EVIDENCE, OR WHAT IS STILL THIN
Git and GitHub	yes / no	
Lightning Web Components	yes / no	
React	yes / no	
React inside LWC	yes / no	
TypeScript	yes / no	
Apex and the patterns	yes / no	
Security and sharing	yes / no	
Experience Cloud and LWR	yes / no	
Tasks 1–6 completed	which ones	links / screenshots

SAYING NO COSTS YOU NOTHING ON THE 19TH

A "no" now is a week we can still plan around: pairing, a reordered first ticket, a different area to start in. A "yes" that turns out to be a "no" in week two costs the whole team, in a 2.5-month window that has no slack in it. Nobody is going to think less of an honest gap. Report the gap.

EXPLICITLY NOT EXPECTED

~~fflib library API~~

~~Next.js~~

~~Redux~~

~~React Router~~

~~Tailwind~~

~~class components~~

~~Aura~~

~~Webpack~~

~~GraphQL~~

~~React Native~~

~~Server-side rendering~~

Our React is plain React 18 with hooks, bundled with esbuild, styled with hand-written CSS. There is no router, no global state library and no CSS framework. Time spent on any of the above is time not spent on the list that matters, so it is stated here rather than left to each person to guess.

IF YOU WANT A ROUTE

Optional, and nobody will ask which of these you did. One workable ordering for the days remaining, if you would rather not design your own:

1. **Git.** Manufacture three merge conflicts deliberately and resolve them. Rebase onto a moved base.
2. **TypeScript.** Write a small module with a generic in it. Make `tsc --noEmit` pass under `strict` with no `any`.
3. **React.** Build a table against a fake async API that does the sorting and paging itself, so you practise asking a server rather than sorting an array.
4. **The bridge.** Clone the starter. Tasks 1 to 3. This is the day that matters most.
5. **Apex.** Tasks 4 and 5. Read the controller and say what makes its dynamic SOQL safe.
6. **Site and tests.** Task 6. Then fill in the table above and send it.

A fuller version of this route, with a drill and a completion check for each day, is available if you want it – ask, and it will be sent. It is a convenience, not the standard. The standard is the list above.